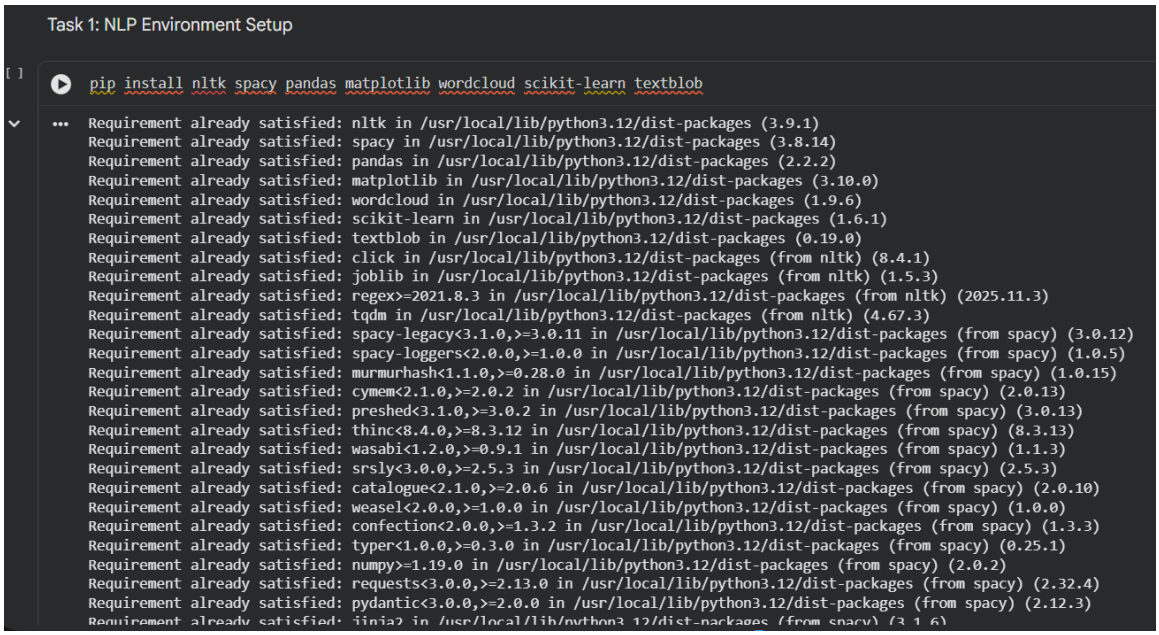
Natural Language Processing (NLP) Fundamentals

Task 1: NLP Environment Setup

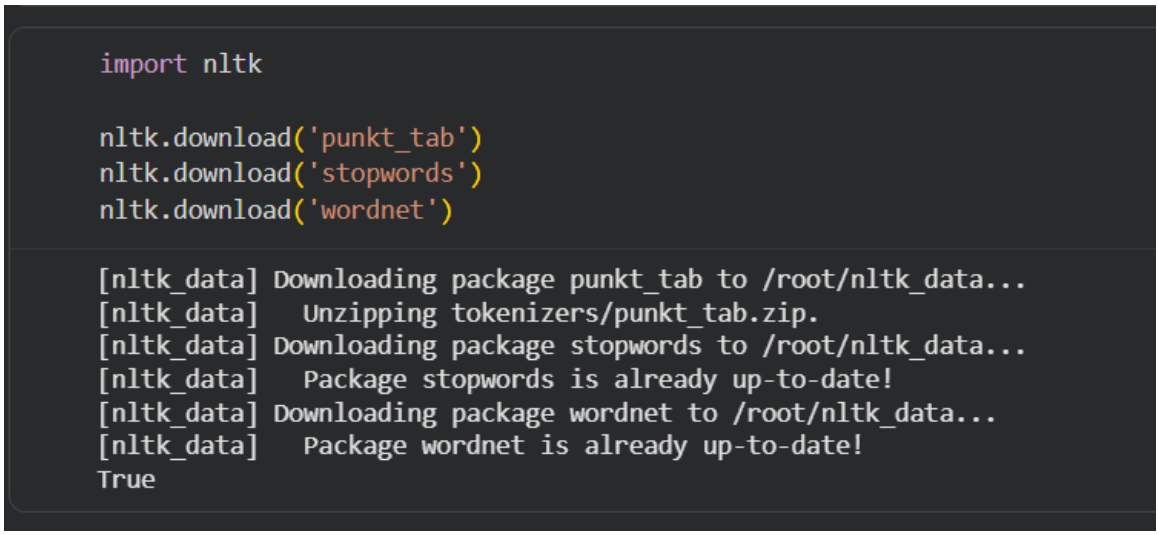
The objective is to prepare the development environment by installing NLP libraries and resources. These tools provide capabilities for text preprocessing, tokenization, linguistic analysis, machine learning, and visualization.



Task 1: NLP Environment Setup

```
pip install nltk spacy pandas matplotlib wordcloud scikit-learn textblob
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)
Requirement already satisfied: spacy in /usr/local/lib/python3.12/dist-packages (3.8.14)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: wordcloud in /usr/local/lib/python3.12/dist-packages (1.9.6)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: textblob in /usr/local/lib/python3.12/dist-packages (0.19.0)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.4.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk) (4.67.3)
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.0.12)
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.0.5)
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.0.15)
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.13)
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.0.13)
Requirement already satisfied: thinc<8.4.0,>=8.3.12 in /usr/local/lib/python3.12/dist-packages (from spacy) (8.3.13)
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.1.3)
Requirement already satisfied: srsly<3.0.0,>=2.5.3 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.5.3)
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.10)
Requirement already satisfied: weasel<2.0.0,>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.0.0)
Requirement already satisfied: confection<2.0.0,>=1.3.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.3.3)
Requirement already satisfied: typer<1.0.0,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (0.25.1)
Requirement already satisfied: numpy>=1.19.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.2)
Requirement already satisfied: requests<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.32.4)
Requirement already satisfied: pydantic<3.0.0,>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.12.3)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.1.6)
```



```
import nltk

nltk.download('punkt_tab')
nltk.download('stopwords')
nltk.download('wordnet')
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
True
```

Task 2: Text Cleaning

Text cleaning is the process of removing unwanted information from raw text. Typical operations include converting text to lowercase, removing URLs, email addresses, phone numbers, special characters, and extra spaces. Clean data improves the quality of NLP models and analytics.

Task 2: Text Cleaning

```
import re
text = """Hello Everyone!!!

Welcome to AI/ML Training.

Contact us at support@company.com

Visit https://company.com

Call 9876543210"""

# Lowercase
text = text.lower()

#Removing Urls
text = re.sub(r'https?://\S+', '',text)

#Removing Email Address
text = re.sub(r'\S+@\S+','',text)

#Removing phone numbers
text = re.sub(r'\d{10}', '',text)

#Removing special characters
text = re.sub(r'[^a-zA-Z\s]','',text)

#Removing extra space
text = re.sub(r'\s+', ' ', text).strip()

print("Cleaned Text:")
print(text)
```

```
Cleaned Text:
hello everyone welcome to aiml training contact us at visit call
```

Task 3: Tokenization

Tokenization divides text into smaller units such as words or sentences. It is one of the first steps in NLP pipelines and enables further analysis such as frequency counting, classification, and information extraction.

Task 3: Tokenization

```
import nltk
from nltk.tokenize import word_tokenize, sent_tokenize

text = "Artificial Intelligence is transforming software development rapidly."

word_tokens = word_tokenize(text)
sentence_tokens = sent_tokenize(text)

print("Word Tokens:")
print(word_tokens)

print("Sentence Tokens:")
print(sentence_tokens)
```

```
Word Tokens:
['Artificial', 'Intelligence', 'is', 'transforming', 'software', 'development', 'rapidly', '.']
Sentence Tokens:
['Artificial Intelligence is transforming software development rapidly.']
```

Task 4: Stop Word Removal

Stop words are common words that often carry little semantic value in analysis. Removing them reduces noise and allows algorithms to focus on meaningful terms within a document.

```
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

text ="Artificial Intelligence is one of the most important technologies in the world."

stop_words = set(stopwords.words('english'))
tokens = word_tokenize(text)
filtered_words = [word for word in tokens if word.lower() not in stop_words]

print("Original Text:")
print(text)

print("\nProcessed Text:")
print(" ".join(filtered_words))
```

```
Original Text:
Artificial Intelligence is one of the most important technologies in the world.

Processed Text:
Artificial Intelligence one important technologies world .
```

Task 5: Stemming and Lemmatization

Both techniques reduce words to their base form. Stemming uses rule-based truncation, while lemmatization applies linguistic knowledge to produce valid dictionary forms. Lemmatization is generally more accurate and meaningful.

Task 5: Stemming & Lemmatization

+ Code

```
from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer

words = [
"running",
"playing",
"studies",
"better",
"cars"
]
stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()

print("Word\t\tStem\t\tLemma")

for word in words:
  stem = stemmer.stem(word)
  lemma = lemmatizer.lemmatize(word)
  print(f"{words}\t\t{stem}\t\t{lemma}")
```

```
Word            Stem            Lemma
['running', 'playing', 'studies', 'better', 'cars']             run             running
['running', 'playing', 'studies', 'better', 'cars']             play            playing
['running', 'playing', 'studies', 'better', 'cars']             studi           study
['running', 'playing', 'studies', 'better', 'cars']             better          better
['running', 'playing', 'studies', 'better', 'cars']             car             car
```

Task 6: Text Vectorization

Text vectorization converts textual data into numerical representations. Common approaches include Bag of Words and TF-IDF. These methods are essential for machine learning algorithms that require numerical input.

Task 6: Text Vectorization

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.feature_extraction.text import TfidfVectorizer

documents = [
    "AI is transforming businesses",
    "Machine learning powers AI",
    "Deep learning is a subset of AI",
    "AI applications are growing rapidly"
]

# Bags of words
bow = CountVectorizer()
bow_matrix = bow.fit_transform(documents)

print("=== Bag of Words ===")
print("Vocabulary:")
print(bow.vocabulary_)
print("\n Matrix Shape:")
print(bow_matrix.shape)
print("\n Features Names:")
print(bow.get_feature_names_out())

#TF-IDF
tfidf = TfidfVectorizer()
tfidf_matrix = tfidf.fit_transform(documents)

print("\n=== TF-IDF ===")
print("Vocabulary:")
print(tfidf.vocabulary_)
print("\n Matrix Shape:")
print(tfidf_matrix.shape)
print("\n Features Names:")
print(tfidf.get_feature_names_out())
```

```
=== Bag of Words ===
Vocabulary:
{'ai': 0, 'is': 6, 'transforming': 13, 'businesses': 3, 'machine': 8, 'learning': 7, 'powers': 10, 'deep': 4, 'subset': 12, 'of': 9, 'applications': 1, 'are': 2, 'growing': 5, 'rapidly': 11}

 Matrix Shape:
(4, 14)

 Features Names:
['ai' 'applications' 'are' 'businesses' 'deep' 'growing' 'is' 'learning'
 'machine' 'of' 'powers' 'rapidly' 'subset' 'transforming']

=== TF-IDF ===
Vocabulary:
{'ai': 0, 'is': 6, 'transforming': 13, 'businesses': 3, 'machine': 8, 'learning': 7, 'powers': 10, 'deep': 4, 'subset': 12, 'of': 9, 'applications': 1, 'are': 2, 'growing': 5, 'rapidly': 11}

 Matrix Shape:
(4, 14)

 Features Names:
['ai' 'applications' 'are' 'businesses' 'deep' 'growing' 'is' 'learning'
 'machine' 'of' 'powers' 'rapidly' 'subset' 'transforming']
```

Task 7: Sentiment Analysis

Sentiment analysis determines the emotional tone of text. Texts are commonly classified as positive, negative, or neutral. Applications include customer feedback analysis, social media monitoring, and product reviews.

Task 7: Sentiment Analysis

```python
from textblob import TextBlob
reviews =  [
    "This course is amazing",
    "The training is excellent",
    "The project is difficult",
    "The application is terrible"
]
print("=== Sentiment Analysis ===")
for review in reviews:

  polarity = TextBlob(review).sentiment.polarity

  if polarity > 0.3:
    sentiment = "Positive"
  elif polarity < -0.5:
    sentiment = "Negative"
  else:
    sentiment = "Neutral"

  print(f"Review: {review}")
  print(f"Sentiment: {sentiment}")
  print("-"*40)
```

```
=== Sentiment Analysis ===
Review: This course is amazing
Sentiment: Positive
----------------------------------------
Review: The training is excellent
Sentiment: Positive
----------------------------------------
Review: The project is difficult
Sentiment: Neutral
----------------------------------------
Review: The application is terrible
Sentiment: Negative
----------------------------------------
```

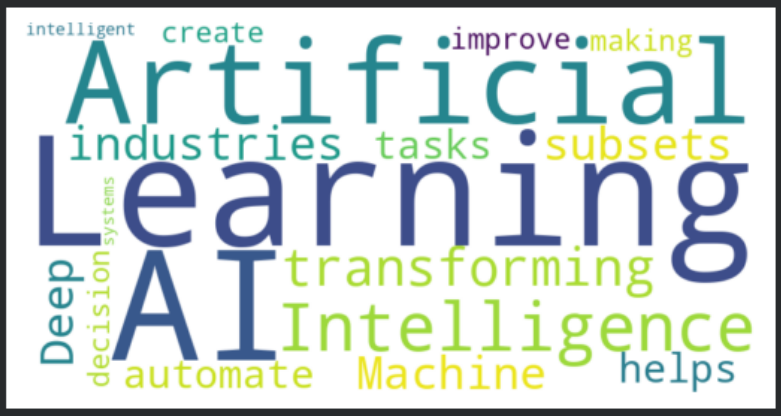
Task 8: Word Cloud Visualization

A word cloud visually represents word frequency. Frequently occurring terms appear larger than less frequent ones. It provides a quick understanding of dominant themes within a text corpus.

Task 8: Word Cloud Visualization

```
from wordcloud import WordCloud
import matplotlib.pyplot as plt
text = """
Artificial Intelligence is transforming industries.
Machine Learning and Deep Learning are subsets of AI.
AI helps automate tasks, improve decision making,
and create intelligent systems.
"""
wordcloud = WordCloud(
    width = 800,
    height = 400,
    background_color = 'white'
).generate(text)

plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.savefig('wordcloud.png')
plt.show()
```



Task 9: Mini NLP Project

This mini-project analyzes resumes to identify important keywords. The system cleans text, removes irrelevant words, extracts significant terms, and generates a keyword frequency report. Such systems are useful in recruitment and applicant tracking systems.

Task 9: Mini NLP Project

```
from collections import Counter
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
import re

with open("/content/sample_data/resume.txt","r",encoding = "utf-8")as file:
  text = file.read()

#cleaning
text = text.lower()
text = re.sub(r'[^a-zA-Z\s]','',text)

#Stopwords Remove
stop_words = set(stopwords.words('english'))
filtered = [
    word for word in word_tokenize(text) if word.lower() not in stop_words
]
# Top keywords
keywords_freq = Counter(filtered)
top_keywords = keywords_freq.most_common(20)

with open("keywords_report.txt",'w')as report:
  report.write("Top 20 Resume Keywords:\n")

  for word, count in top_keywords:
    report.write(f"{word}:{count}\n")
print("Keyword_report.txt generated successfully")
```

```
Keyword_report.txt generated successfully
```

Bonus Challenge: AI Chat Preprocessor

This project builds a complete NLP preprocessing pipeline for chat messages. It includes cleaning, tokenization, stop word removal, lemmatization, keyword extraction, and report generation.

Bonus Challenge - AI Chat Preprocessor

```python
import re
from collections import Counter
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

chat = input("Enter chat message:")

#clean text
chat = chat.lower()
chat = re.sub(r'^a-zA-Z\s','',chat)

#tokenize
tokens = word_tokenize(chat)

#stopwords
stop_words = set(stopwords.words('english'))

filtered = [
    word for word in tokens
    if word not in stop_words
]

lemmas = [
    lemmatizer.lemmatize(word)
    for word in filtered
]

#keywords
keywords = Counter(lemmas)
```

```python
#keywords
keywords = Counter(lemmas)

print('\n cleaned Text:')
print(chat)
print("\n Tokens:")
print(tokens)
print("\n After Stop Word Removal:")
print(filtered)
print("\n After Lemmatization:")
print(lemmas)
print("\n Top Keywords:")
print(keywords.most_common(10))

#Export Report
with open("chat_report.txt","w")as file:
  file.write("AI chat preprocessor Report\n\n")
  file.write(f"Tokens: {tokens}\n\n")
  file.write(f"Filtered Words: {filtered}\n\n")
  file.write(f"Lemmatized Words: {lemmas}\n\n")
  file.write(f"Top Keywords: {keywords.most_common(10)}")
print("\n chat_report.txt generated Successfully")
```

```
Enter chat message:Artificial Intelligence is transforming industries.

 cleaned Text:
artificial intelligence is transforming industries.

 Tokens:
['artificial', 'intelligence', 'is', 'transforming', 'industries', '.']

 After Stop Word Removal:
['artificial', 'intelligence', 'transforming', 'industries', '.']

 After Lemmatization:
['artificial', 'intelligence', 'transforming', 'industry', '.']

 Top Keywords:
[('artificial', 1), ('intelligence', 1), ('transforming', 1), ('industry', 1), ('.', 1)]

 chat_report.txt generated Successfully
```